

JSP 模式和 JDBC 高级

今日内容介绍

- ◆ 案例：使用 MVC 完成转账

今日内容学习目标

- ◆ 了解 MySQL 事务操作
- ◆ 掌握 JDBC 事务操作

第1章 案例：使用MVC完成转账

1.1 需求

- 完成转账功能

1.2 相关知识点

1.2.1 事务管理

1.2.1.1 事务概述

- 事务指的是逻辑上的一组操作,组成这组操作的各个单元要么全都成功,要么全都失败.
- 事务作用：保证在一个事务中多次操作要么全都成功,要么全都失败.

1.2.1.2 mysql 事务操作

sql 语句	描述
--------	----

start transaction;	开启事务
commit;	提交事务
rollback;	回滚事务

- 准备数据

```
# 创建一个表: 账户表.
create database day19_db;

# 使用数据库
use day19_db;

# 创建账号表
create table account(
    id int primary key auto_increment,
    name varchar(20),
    money double
);

# 初始化数据
insert into account values (null,'jack',10000);
insert into account values (null,'rose',10000);
insert into account values (null,'tom',10000);
```

- 操作:

- MYSQL 中可以有两种方式进行事务的管理:

- ◆ 自动提交: MySQL 默认自动提交。及执行一条 sql 语句提交一次事务。
- ◆ 手动提交: 先开启, 再提交

- 方式 1: 手动提交

```
start transaction;
update account set money=money-1000 where name='守义';
update account set money=money+1000 where name='凤儿';
commit;
#或者
rollback;
```

- 方式 2: 自动提交, 通过修改 mysql 全局变量 “autocommit” 进行控制

```
show variables like '%commit%';
```

Variable_name	Value
autocommit	ON
innodb_commit_concurrency	0
innodb_flush_log_at_trx_commit	1

* 设置自动提交的参数为 OFF:

```
set autocommit = 0; -- 0:OFF 1:ON
```

- 扩展: Oracle 数据库事务不自动提交

1.2.1.3 JDBC 事务操作

Connection 对象的方法名	描述
conn.setAutoCommit(false)	开启事务
conn.commit()	提交事务
conn.rollback()	回滚事务

```
//事务模板代码
public void demo01() throws SQLException{
    // 获得连接
    Connection conn = null;

    try {
        // #1 开始事务
        conn.setAutoCommit(false);

        // .... 加钱 , 减钱

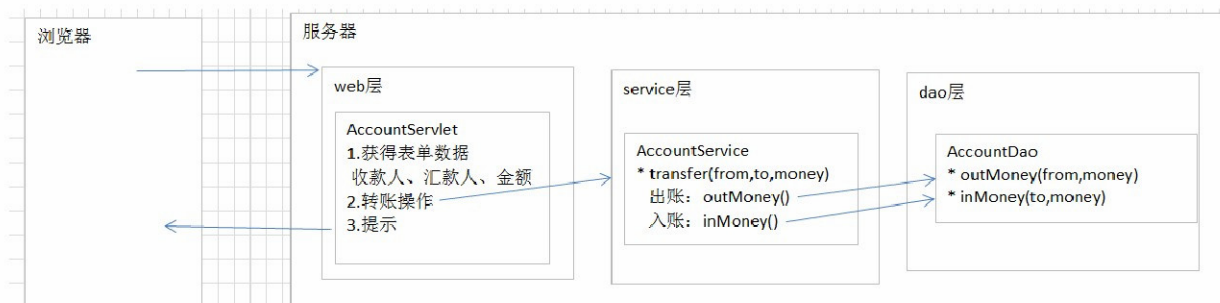
        // #2 提交事务
        conn.commit();
    } catch (Exception e) {
        // #3 回滚事务
        conn.rollback();
    } finally{
        // 释放资源
        conn.close();
    }
}
```

1.2.1.4 DBUtils 事务操作

Connection 对象的方法名	描述
conn.setAutoCommit(false)	开启事务
new QueryRunner()	创建核心类, 不设置数据源(手动管理连接)
query(conn, sql, handler, params) 或 update(conn, sql, params)	手动传递连接
DbUtils.commitAndClose(conn) 或 DbUtils.rollbackAndClose(conn)	提交并关闭连接 回滚并关闭连接

1.3 分析

1.3.1 案例分析



1.3.2 步骤分析：

- 步骤一：设计页面三个文本框。
- 步骤二：提交到 Servlet
 - 接收参数：
 - 封装参数：
 - 调用业务层处理数据
 - 页面跳转

1.4 代码实现

- 步骤 1：创建 index.jsp 页面，设计转账页面

```
<h1>转账的页面</h1>
<form action="${pageContext.request.contextPath}/AccountServlet" method="post">
    <table border="1" width="400">
        <tr>
            <td>付款人:</td>
            <td><input type="text" name="from"/></td>
        </tr>
        <tr>
            <td>收款人:</td>
            <td><input type="text" name="to"/></td>
        </tr>
        <tr>
            <td>转账金额:</td>
            <td><input type="text" name="money"/></td>
        </tr>
    </table>
</form>
```

```
</tr>
<tr>
    <td colspan="2"><input type="submit" value="转账"/></td>
</tr>
</table>
</form>
```

● 步骤 2: 编写 AccountServlet

```
public class AccountServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        /**
         * 1.接收数据:
         * 2.封装数据:
         * 3.调用业务层:
         * 4.页面跳转作出响应:
         */
        request.setCharacterEncoding("UTF-8");
        // 接收数据:
        String from = request.getParameter("from");
        String to = request.getParameter("to");
        double money = Double.parseDouble(request.getParameter("money"));
        // 调用业务层:
        AccountService accountService = new AccountService();
        accountService.transfer(from,to,money);
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```

● 步骤 3: 编写 AccountService

```
public class AccountService {

    /**
     * 业务层转账的方法:
     * @param from :付款人
```

```
    * @param to      :收款人
    * @param money :转账金额
    */
    public void transfer(String from, String to, double money) {
        // 调用 DAO:
        AccountDao accountDao = new AccountDao();
        try {
            accountDao.outMoney(from, money);
            // int d = 1/0;
            accountDao.inMoney(to, money);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

步骤 4: 编写 AccountDao

```
public class AccountDao {

    /**
     * 付款的方法
     * @param name
     * @param money
     * @throws SQLException
     */
    public void outMoney(String name, double money) throws SQLException {
        Connection conn = null;
        PreparedStatement pstmt = null;
        try {
            // 获得连接:
            conn = JDBCUtils.getConnection();
            // 编写一个 SQL:
            String sql = "update account set money = money-? where name=?";
            // 预编译 SQL:
            pstmt = conn.prepareStatement(sql);
            // 设置参数:
            pstmt.setDouble(1, money);
            pstmt.setString(2, name);
            // 执行 SQL:
            pstmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



```
        }finally{
            pstmt.close();
            conn.close();
        }
    }

    /**
     * 收款的方法
     * @param name
     * @param money
     * @throws SQLException
     */
    public void inMoney(String name,double money) throws SQLException{
        Connection conn = null;
        PreparedStatement pstmt = null;
        try{
            // 获得连接:
            conn = JDBCUtils.getConnection();
            // 编写一个 SQL:
            String sql = "update account set money = money+? where name=?";
            // 预编译 SQL:
            pstmt = conn.prepareStatement(sql);
            // 设置参数:
            pstmt.setDouble(1, money);
            pstmt.setString(2, name);
            // 执行 SQL:
            pstmt.executeUpdate();
        }catch(Exception e){
            e.printStackTrace();
        }finally{
            pstmt.close();
            conn.close();
        }
    }
}
```

1.5 事务管理：传递 Connection

- 修改 service 和 dao，service 将 connection 传递给 dao，dao 不需要自己获得连接

1.5.1 service 层

```
public void transfer(String outUser,String inUser,int money){
    Connection conn =null;
    try{
        //1 获得连接
        conn = JdbcUtils.getConnection();
        //2 开启事务
        conn.setAutoCommit(false);

        accountDao.outMoney(conn,outUser, money);
        //断电
        //int i = 1 / 0;
        accountDao.inMoney(conn,inUser, money);

        //3 提交事务
        conn.commit();
    } catch (Exception e) {
        try {
            //回顾
            if (conn != null) {
                conn.rollback();
            }
        } catch (Exception e2) {
        }
        throw new RuntimeException(e);
    } finally{
        JdbcUtils.closeResource(conn, null, null);
    }
}
```

1.5.2 dao 层

```
/**
 * 汇款
 * @param outUser 汇款人
 * @param money -
 */
public void outMoney(Connection conn, String outUser , int money){
    //Connection conn = null;
    PreparedStatement psmt = null;
```



```
ResultSet rs = null;
try {
    //1 获得连接
    //conn = JdbcUtils.getConnection();
    //2 准备 sql 语句
    String sql = "update account set money = money - ? where username = ?";
    //3 预处理
    pstmt = conn.prepareStatement(sql);
    //4 设置实际参数
    pstmt.setInt(1, money);
    pstmt.setString(2, outUser);
    //5 执行
    int r = pstmt.executeUpdate();
    System.out.println(r);
} catch (Exception e) {
    throw new RuntimeException(e);
} finally{
    //6 释放资源
    JdbcUtils.closeResource(null, pstmt, rs);
}
}
/**
 * 收款
 * @param inUser 收款人
 * @param money +
 */
public void inMoney(Connection conn,String inUser , int money){
    //Connection conn = null;
    PreparedStatement pstmt = null;
    ResultSet rs = null;
    try {
        //1 获得连接
        //conn = JdbcUtils.getConnection();
        //2 准备 sql 语句
        String sql = "update account set money = money + ? where username = ?";
        //3 预处理
        pstmt = conn.prepareStatement(sql);
        //4 设置实际参数
        pstmt.setInt(1, money);
        pstmt.setString(2, inUser);
        //5 执行
        int r = pstmt.executeUpdate();
        System.out.println(r);
    } catch (Exception e) {
```

```
        throw new RuntimeException(e);
    } finally{
        //6 释放资源
        JdbcUtils.closeResource(null, psmt, rs);
    }
}
```

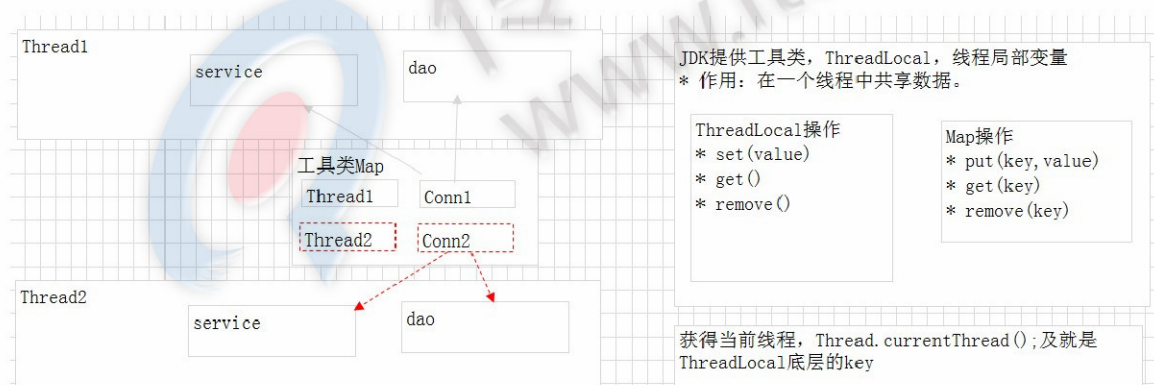
1.6 提高：ThreadLocal

1.6.1 分析

在“事务传递参数版”中，我们必须修改方法的参数个数，传递链接，才可以完成整个事务操作。如果不传递参数，是否可以完成？在 JDK 中给我们提供了一个工具类：**ThreadLocal**，此类可以在一个线程中共享数据。

1.6.2 相关知识：ThreadLocal

`java.lang.ThreadLocal` 该类提供了线程局部 (thread-local) 变量，用于在当前线程中共享数据。`ThreadLocal` 工具类底层就是一个 `Map`，`key` 存放的当前线程，`value` 存放需要共享的数据。



1.6.3 实现

1.6.3.1 工具类 JDBCUtils

```
//连接池
private static ComboPooledDataSource dataSource = new
ComboPooledDataSource("itcast");

//给当前线程绑定 连接
```

```
private static ThreadLocal<Connection> local = new ThreadLocal<Connection>();

/**
 * 获得连接
 * @return
 */
public static Connection getConnection(){
    try {
        // #1 从当前线程中， 获得已经绑定的连接
        Connection conn = local.get();
        if(conn == null){
            // #2 第一次获得， 绑定内容 - 从连接池获得
            conn = dataSource.getConnection();
            // #3 将连接存 ThreadLocal
            local.set(conn);
        }
        return conn; // 获得连接
    } catch (Exception e) {
        // 将编译时异常 转换 运行时， 以后开发中 运行时异常使用比较多的。
        // * 此处可以编写自定义异常。
        throw new RuntimeException(e);
        // * 类与类之间 进行数据交换时， 可以使用 return 返回值。 也可以自定义异常返回值， 调用者 try{} catch(e){ e.getMessage() 获得需要的数据 }
        // throw new MyConnectionException(e);
    }
}
```

1.6.3.2 service 层

```
public void transfer(String outUser,String inUser,int money){

    Connection conn =null;
    try{
        // 1 获得连接
        conn = JdbcUtils.getConnection();
        // 2 开启事务
        conn.setAutoCommit(false);

        accountDao.out(outUser, money);
        // 断电
        // int i = 1 / 0;
        accountDao.in(inUser, money);
    }
```

```
        //3 提交事务
        conn.commit();
    } catch (Exception e) {
        try {
            //回顾
            if (conn != null) {
                conn.rollback();
            }
        } catch (Exception e2) {
        }
        throw new RuntimeException(e);
    } finally{
        JdbcUtils.closeResource(conn, null, null);
    }
}
```

1.6.3.3 dao 层

```
/**
 * 汇款
 * @param outUser 汇款人
 * @param money -
 */
public void out(String outUser , int money){
    Connection conn = null;
    PreparedStatement psmt = null;
    ResultSet rs = null;
    try {
        //1 获得连接
        conn = JdbcUtils.getConnection();
        //2 准备 sql 语句
        String sql = "update account set money = money - ? where username = ?";
        //3 预处理
        psmt = conn.prepareStatement(sql);
        //4 设置实际参数
        psmt.setInt(1, money);
        psmt.setString(2, outUser);
        //5 执行
        int r = psmt.executeUpdate();
        System.out.println(r);
    } catch (Exception e) {
```

```
        throw new RuntimeException(e);
    } finally{
        //6 释放资源--不能关闭连接
        JdbcUtils.closeResource(null, psmt, rs);
    }
}
/**
 * 收款
 * @param inUser 收款人
 * @param money +
 */
public void in(String inUser , int money){
    Connection conn = null;
    PreparedStatement psmt = null;
    ResultSet rs = null;
    try {
        //1 获得连接
        conn = JdbcUtils.getConnection();
        //2 准备 sql 语句
        String sql = "update account set money = money + ? where username = ?";
        //3 预处理
        psmt = conn.prepareStatement(sql);
        //4 设置实际参数
        psmt.setInt(1, money);
        psmt.setString(2, inUser);
        //5 执行
        int r = psmt.executeUpdate();
        System.out.println(r);
    } catch (Exception e) {
        throw new RuntimeException(e);
    } finally{
        //6 释放资源--注意：不能关闭链接
        JdbcUtils.closeResource(null, psmt, rs);
    }
}
```

1.7 案例总结：事务总结

1.7.1 事务特性：ACID

- 原子性（Atomicity）原子性是指事务是一个不可分割的工作单位，事务中的操作要么都发生，

要么都不发生。

- 一致性（Consistency）事务前后数据的完整性必须保持一致。
- 隔离性（Isolation）事务的隔离性是指多个用户并发访问数据库时，一个用户的事务不能被其它用户的事务所干扰，多个并发事务之间数据要相互隔离。
- 持久性（Durability）持久性是指一个事务一旦被提交，它对数据库中数据的改变就是永久性的，接下来即使数据库发生故障也不应该对其有任何影响。

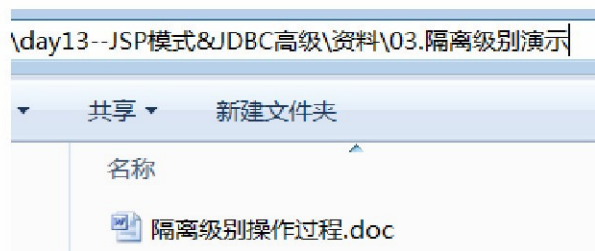
1.7.2 并发访问问题

如果不考虑隔离性，事务存在 3 中并发访问问题。

1. 脏读：一个事务读到了另一个事务未提交的数据。
2. 不可重复读：一个事务读到了另一个事务已经提交(update)的数据。引发另一个事务，在事务中的多次查询结果不一致。
3. 虚读 /幻读：一个事务读到了另一个事务已经提交(insert)的数据。导致另一个事务，在事务中多次查询的结果不一致。

1.7.3 隔离级别：解决问题

- 数据库规范规定了 4 种隔离级别，分别用于描述两个事务并发的所有情况。
 1. read uncommitted 读未提交，一个事务读到另一个事务没有提交的数据。
 - a) 存放：3 个问题（脏读、不可重复读、虚读）。
 - b) 解决：0 个问题
 2. read committed 读已提交，一个事务读到另一个事务已经提交的数据。
 - a) 存放：2 个问题（不可重复读、虚读）。
 - b) 解决：1 个问题（脏读）
 3. repeatable read :可重复读，在一个事务中读到的数据始终保持一致，无论另一个事务是否提交。
 - a) 存放：1 个问题（虚读）。
 - b) 解决：2 个问题（脏读、不可重复读）
 4. serializable 串行化，同时只能执行一个事务，相当于事务中的单线程。
 - a) 存放：0 个问题。
 - b) 解决：1 个问题（脏读、不可重复读、虚读）
- 安全和性能对比
 - 安全性：serializable > repeatable read > read committed > read uncommitted
 - 性能：serializable < repeatable read < read committed < read uncommitted
- 常见数据库的默认隔离级别：
 - MySql: repeatable read
 - Oracle: read committed
- 隔离级别演示参考：资料/03.隔离级别演示/《隔离级别操作过程.doc》[增强内容,了解]



第2章 总结

